The bisect option for Minitest and RSpec gives you a minimal set of specs that reproduces the error so that you can run the minimal set from the command line and reproduce the problem efficiently. Once the minimal set is identified, it's much easier to find the problem.

The two commands work almost identically, which is not surprising since RSpec explicitly cribbed the feature from Minitest. The Minitest version does require a gem, minitest-bisect, to be installed.

To make this work, you need a test suite that has an intermittent failure that you believe is due to test ordering, and you need a seed value that produces a failure. In RSpec, if you're not already running the tests in a random order, add the line `--order rand` to the `.rspec` file. Each run will then tell you what seed value RSpec used for the randomization `Randomized with seed 57180`. You can then reproduce the exact ordering with `rspec --seed 57180`. Minitest behaves similarly, except that the random ordering is the default behavior.

Once you have a seed that consistently produces an ordering with a test failure, you trigger the bisect. In RSpec that's done with the following:

```
$ rspec --seed 57180 --bisect
```

In Minitest it looks like this:

```
$ minitest_bisect --seed 57180 test/**/_*test.rb
```

In either case, replace the seed value with whatever your tests accept. In Minitest, the final argument is a file glob selecting test files. If you know that the tests are in a smaller subset already, you can specify that instead.

In both cases, the bisect tool will run tests for a while. Possibly a long while. But if a test failure is really caused by test ordering, both tools will end up with a minimal set of individual tests (not test files) that will cause it, and a command-line command suitable for cutting and pasting that will run those tests.

The test bisect is tricky. It will not help you with some kinds of intermittent failures (for example, Capybara timing issues), but like Git's bisect, when it works, it can save you hours.

Pry

Once upon a time, I was a fan of development environments that had big fancy symbolic debuggers that let you set breakpoints and watch variables and step through the code.

What can I say? I was young and programming in Java, a language that had large integrated development environments.